



Universidade Tecnológica Federal do Paraná

Câmpus Cornélio Procópio

Curso de Engenharia de Software

Disciplina: Estrutura de Dados 1

LISTAS ENCADEADAS

Trabalho escrito

Integrante

Alan Diogo Ribeiro de Carvalho

Professora

Monique Oliveira

Cornélio Procópio

Setembro de 2026

1 Introdução

Na disciplina de Estrutura de Dados 1, o estudo de ponteiros em C mostrou que uma variável pode guardar o endereço de outra e que, por meio desse endereço, é possível ler e alterar o conteúdo apontado. A lista encadeada é a primeira estrutura em que esse mecanismo deixa de ser um exercício isolado e passa a organizar uma sequência de dados.

Em um vetor, os elementos ficam em posições vizinhas de memória e o acesso é feito por índice. Na lista encadeada, cada elemento fica em um bloco próprio, obtido com alocação dinâmica, e a ordem da sequência é dada por ponteiros. O primeiro elemento é alcançado por um ponteiro de início. Os demais são alcançados seguindo o campo que aponta para o próximo nó.

Este trabalho explica o que é uma lista encadeada, por que ela é usada, qual conceito já estudado sustenta essa estrutura, como um nó é organizado e como um valor é inserido no início da lista. O exemplo está escrito em C, na mesma linguagem das aulas de ponteiros. O foco é a lista simplesmente encadeada, em que cada nó conhece apenas o sucessor.

2 Desenvolvimento

2.1 O que são listas encadeadas

Uma lista encadeada é uma sequência de nós ligados por ponteiros. Cada nó guarda um dado e o endereço do nó seguinte. O último nó não possui sucessor: o ponteiro dele recebe o valor *NULL*, que marca o fim da lista. Quando a lista está vazia, o próprio ponteiro de início vale *NULL*.

A posição física dos nós na memória não precisa ser consecutiva. O nó que guarda o valor 10 pode estar longe do nó que guarda o valor 20. A ordem lógica 10, depois 20, existe porque o campo de ligação do primeiro aponta para o segundo. Por isso a lista é chamada de encadeada: a cadeia é feita de endereços, não de vizinhança na memória.

Há variações. Na lista simplesmente encadeada, a ligação segue um único sentido. Na lista duplamente encadeada, cada nó também aponta para o anterior. Na lista circular, o último nó aponta de volta para o primeiro. O procedimento pedido neste trabalho, inserir no início, é descrito sobre a lista simplesmente encadeada, que é a forma mais direta de aplicar ponteiros.

```
início → [ 10 | prox ] → [ 20 | prox ] → [ 30 | prox ] → NULL
```

Figura 1 — Lista simplesmente encadeada com três nós.

2.2 Por que usar listas encadeadas

O vetor resolve bem o acesso direto: o elemento da posição *i* é localizado em tempo constante. O custo aparece quando a quantidade de dados muda ou quando a inserção ocorre no começo. Para colocar um valor novo na primeira posição de um vetor, todos os elementos já armazenados precisam ser deslocados uma casa para a direita. Se o vetor foi criado com tamanho fixo e enche, é preciso alocar outro bloco e copiar o conteúdo.

A lista encadeada inverte esse compromisso. Inserir ou remover no início não desloca os nós que já existem: basta ajustar ponteiros. A quantidade de elementos cresce e diminui um nó por vez, com *malloc* e *free*, sem reservar de antemão um tamanho máximo. Isso é útil quando o programa não sabe quantos itens chegarão, como em uma fila de atendimento, no histórico de uma edição ou em uma sequência que só é percorrida do começo ao fim.

O uso tem preço. Não há índice. Para chegar ao décimo elemento, o programa percorre os nove anteriores, seguindo *prox*. Cada nó também gasta memória extra com o ponteiro de ligação, e os nós espalhados pela memória aproveitam pior a cache do processador do que um vetor contíguo. A lista é a estrutura adequada quando a operação frequente é inserir ou remover nas pontas e a travessia é sequencial. O vetor continua melhor quando o programa precisa saltar para uma posição qualquer.

2.3 Conceito já estudado e aplicado na lista

O conceito aplicado na lista encadeada é o ponteiro. Nas aulas de ponteiros, uma variável como *int *p* guarda o endereço de um inteiro. O operador *&* obtém o endereço, e o operador *** acessa o valor que está nesse endereço. Também foi visto que uma função só altera a variável do chamador se receber o endereço dela. Receber uma cópia do valor não muda o original.

Na lista, esse mesmo mecanismo vira a estrutura. O campo *prox* é um ponteiro para outro nó. O ponteiro *inicio* guarda o endereço do primeiro nó. Percorrer a lista é repetir a leitura de *prox* até encontrar *NULL*. Alocar um nó com *malloc* devolve um endereço, que só pode ser guardado em um ponteiro.

A inserção no início usa o caso em que o ponteiro precisa ser alterado dentro de uma função. O ponteiro *lista* está em *main*. A função *inserirInicio* precisa fazer esse ponteiro passar a apontar para o nó novo. Se a função recebesse apenas uma cópia de *lista*, a cópia mudaria e *main* continuaria com o endereço antigo. Por isso o parâmetro é *No **inicio*: um ponteiro para o ponteiro de início. A atribuição **inicio = novo* grava o endereço novo na variável do chamador. É o mesmo raciocínio de passar *int *p* para uma função modificar o inteiro apontado, agora aplicado ao ponteiro que representa a lista.

2.4 Estrutura de um nó

O nó é o registro que representa um elemento da lista. No exemplo deste trabalho ele guarda um inteiro e o endereço do sucessor. Em C, isso é uma estrutura autorreferenciada: o tipo *No*

contém um ponteiro para *No*.

```
typedef struct No {  
    int valor;  
    struct No *prox;  
} No;
```

O campo *valor* é a informação do elemento. Aqui ele é um *int* para manter o exemplo curto. Em outro problema poderia ser uma nota, um nome ou outra estrutura. O campo *prox* não guarda o próximo valor: guarda o endereço do próximo nó. Quando *prox* vale *NULL*, aquele nó é o último.

O nó não contém a lista inteira. Ele conhece apenas o próprio dado e o elo seguinte. A lista, como um todo, é o ponteiro para o primeiro nó mais a cadeia formada pelos campos *prox*. Dois papéis ficam separados: o nó carrega um elemento; o ponteiro de início indica onde a sequência começa.

```
+-----+-----+  
| valor | prox |  
+-----+-----+
```

Figura 2 — Campos de um nó: dado e endereço do sucessor.

2.5 Como inserir um valor no início

Inserir no início coloca o novo elemento antes do atual primeiro nó e atualiza o ponteiro de início. O restante da lista não é percorrido e nenhum valor já armazenado é copiado. A operação tem quatro passos.

1. Alocar um nó novo com *malloc(sizeof(No))*. Se a alocação falhar, o ponteiro devolvido é *NULL* e a inserção deve ser interrompida.
2. Gravar o valor informado no campo *valor* desse nó.
3. Fazer *prox* do nó novo apontar para o início atual. Se a lista estava vazia, esse endereço é *NULL* e o nó novo já nasce como último. Se a lista tinha elementos, o antigo primeiro nó passa a ser o segundo.
4. Fazer o ponteiro de início apontar para o nó novo. A partir daí, quem percorre a lista a partir de *inicio* encontra primeiro o valor recém-inserido.

A ordem dos passos 3 e 4 não pode ser invertida. O endereço do antigo primeiro nó só está guardado em **inicio*. Se esse ponteiro for substituído antes de o nó novo copiá-lo, a cadeia antiga fica sem referência e a lista perde os elementos que já continha.

antes: início → [20 | prox] → [30 | prox] → NULL

depois: início → [10 | prox] → [20 | prox] → [30 | prox] → NULL

Figura 3 — Inserção do valor 10 no início de uma lista que já continha 20 e 30.

A função abaixo recebe o endereço do ponteiro de início e o valor a inserir. O programa de demonstração começa com a lista vazia e insere 30, depois 20 e depois 10. Como cada inserção ocorre no início, a impressão fica *início -> [10] -> [20] -> [30] -> NULL*.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;
    struct No *prox;
} No;

void inserirInicio(No **início, int valor) {
    No *novo = malloc(sizeof(No));

    if (novo == NULL) {
        printf("Falha ao alocar memoria.\n");
        return;
    }

    novo->valor = valor;
    novo->prox = *início;
    *início = novo;
}

void imprimir(No *início) {
    No *atual = início;

    printf("início -> ");

    while (atual != NULL) {
        printf("[%d] -> ", atual->valor);
        atual = atual->prox;
    }

    printf("NULL\n");
}

int main(void) {
    No *lista = NULL;

    inserirInicio(&lista, 30);
    inserirInicio(&lista, 20);
    inserirInicio(&lista, 10);

    imprimir(lista);
    return 0;
}
```

Na primeira chamada, *lista* vale *NULL*. O nó de valor 30 aponta para *NULL* e *lista* passa a apontar para ele. Na segunda, o nó de valor 20 aponta para o nó 30. Na terceira, o nó de valor 10 aponta para o nó 20. O ponteiro *lista*, em *main*, acompanha essas trocas porque a função recebeu *&lista*.

3 Conclusão

A lista encadeada guarda uma sequência fora da contiguidade de um vetor. Cada elemento é um nó, e a ordem existe porque cada nó aponta para o próximo. O ponteiro, conceito já exercitado em C, é o que torna essa ligação possível: ele guarda endereços, permite percorrer a cadeia e, quando passado por endereço, permite que uma função atualize o início da lista no chamador.

Usar a lista compensa quando o programa insere ou remove nas pontas e ainda não conhece a quantidade final de elementos. O acesso por posição deixa de ser direto, e cada nó carrega o custo do ponteiro *prox*. Inserir no início é a operação mais simples dessa estrutura: aloca-se o nó, copia-se o início atual para *prox* e só então o início passa a apontar para o nó novo. Feito nessa ordem, o valor entra na frente da lista e os elementos anteriores permanecem encadeados.

Referências

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

SCHILDT, Herbert. **C completo e total**. 3. ed. São Paulo: Pearson Makron Books, 1997.

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. **Estruturas de dados usando C**. São Paulo: Pearson Makron Books, 1995.

ZIVIANI, Nivio. **Projeto de algoritmos: com implementações em Pascal e C**. 3. ed. São Paulo: Cengage Learning, 2011.